

Capa de red II

Datagrama

Programación y administración de redes

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén

Objetivos

General

Conocer el funcionamiento del protocolo IP en términos generales, las funciones que ofrece y el formato de los datagramas de IPv4 e IPv6

Específicos

- Conocer la **estructura** de los datagramas de IPv4 e IPv6
- Identificar la finalidad de los **campos** de la cabecera de los datagramas
- Reconocer los **tipos de paquete** que pueden incluirse en un datagrama
- Saber cómo se controla el **tiempo de vida** de los datagramas
- Conocer el mecanismo de **fragmentación** de IP

Introducción

El protocolo IP

Es el principal protocolo de la capa de red y tiene la responsabilidad de facilitar la comunicación entre host no directamente conectados, reenviando los datagramas a través de los dispositivos de conmutación

Cómo funciona

1. En el **host de origen** el segmento TCP/UDP se aloja en un datagrama
2. IP se encarga de configurar los **campos de la cabecera** para iniciar la transmisión del datagrama
3. Los **elementos de conmutación**, casi siempre *router*, examinan la cabecera para reenviar el datagrama por la mejor ruta
4. Los *router* también **actualizan** algunos campos del datagrama
5. La capa de red en el **host de destino** extrae del datagrama el segmento y lo entrega a la capa de transporte

Tareas a cargo del protocolo IP

Responsabilidad

Para afrontar el reenvío de los datagramas hasta su destino, que es la principal responsabilidad de IP, este tiene que abordar múltiples tareas

Detalles

- Determinar qué **protocolo de la capa de transporte** envía los datos para entregar el segmento a TCP o UDP en el *host* de destino
- Evitar que un datagrama pueda estar eternamente circulando por Internet si entra en **un bucle entre múltiples equipos** de conmutación
- En caso necesario, **fragmentar el datagrama** para facilitar su transferencia a través de redes con una MTU (*Maximum Transfer Unit*) de menor tamaño
- **Detectar posibles errores** en la transmisión de los datagramas
- Atender otras **peticiones específicas** que el emisor haya solicitado

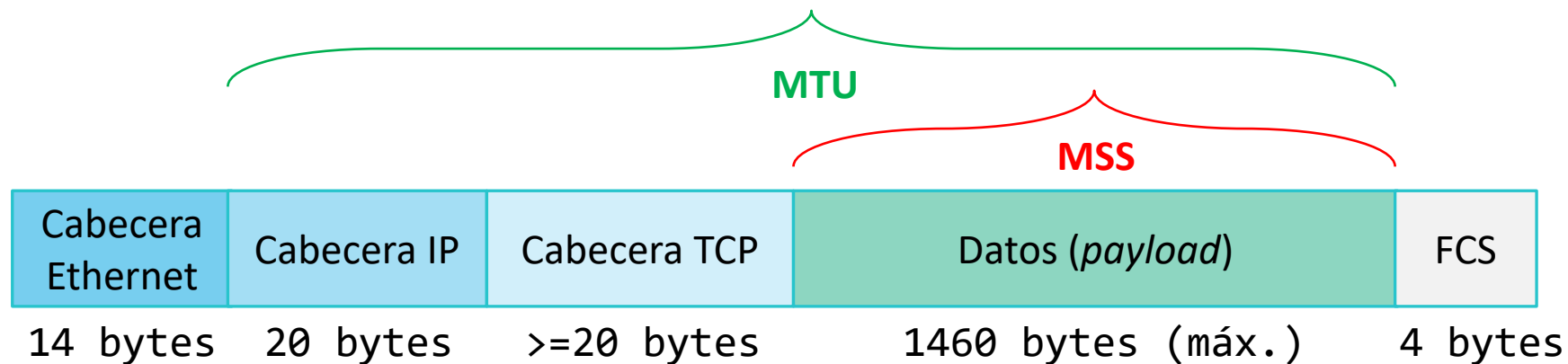
Conceptos de MTU y MSS

Definición

El tamaño máximo del payload de un segmento de la capa de transporte depende del MSS (Maximum Segment Size), un parámetro que depende de la MTU (Maximum Transfer Unit)

Relación entre MTU y MSS:

- El MSS depende del tamaño de la **MTU** que ofrezca la capa de enlace
- Para la capa de enlace de Ethernet la MTU es de 1500 bytes (otras tecnologías tienen MTU distintas)
- **MSS = MTU - Tamaños de cabeceras** (p.e. la cabecera de TCP habitualmente ocupa 20 bytes, al igual que la de IP)

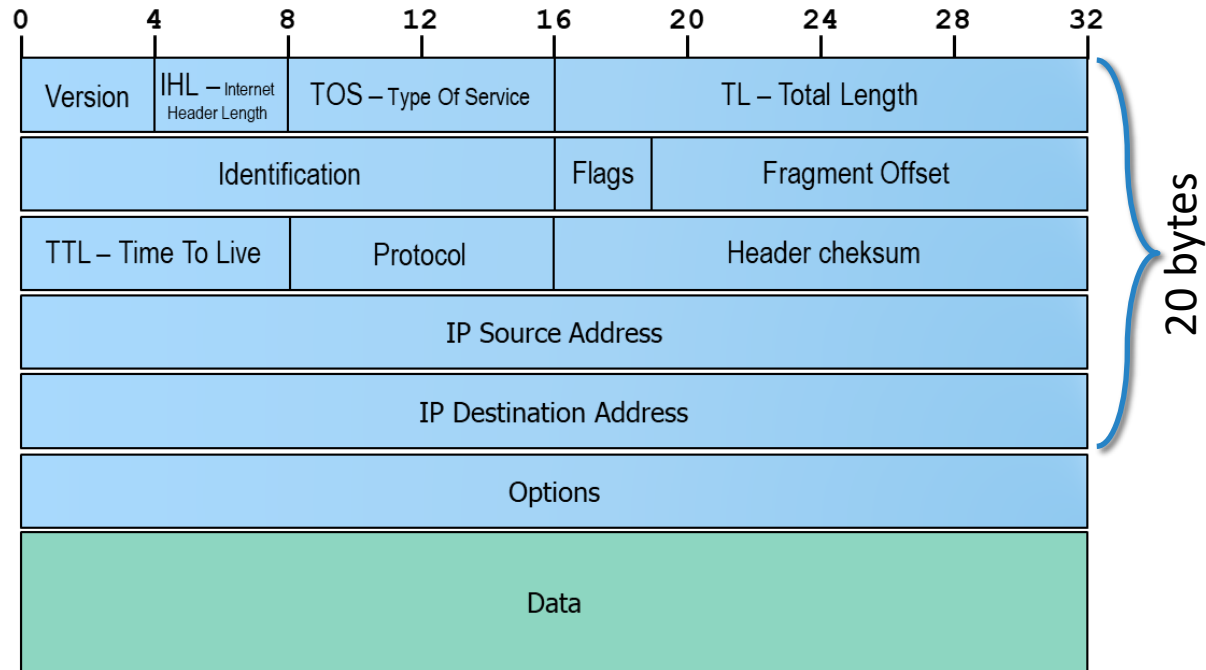


Formato del datagrama

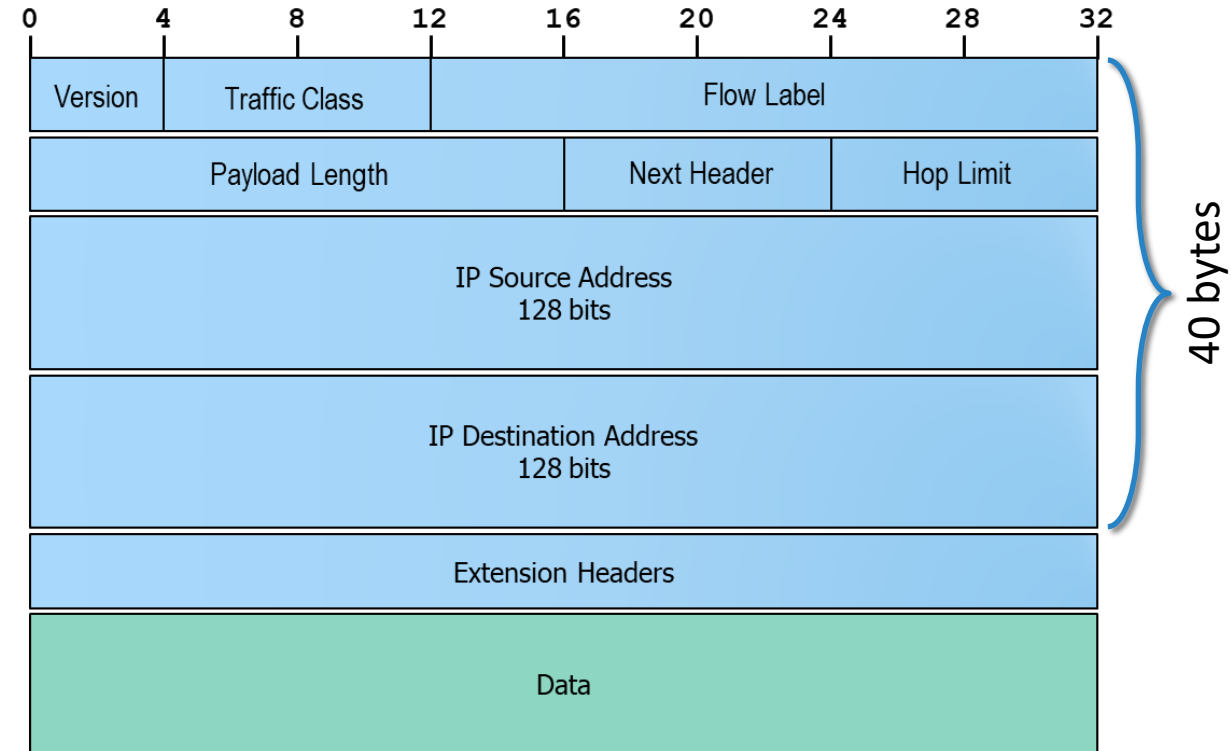


Datagrama de IPv4 frente al de IPv6

IPv4



IPv6



Datagrama de IPv4 frente al de IPv6

Detalles

- IPv4 cuenta con una cabecera de **longitud variable** según que se incluyan o no opciones, de ahí la necesidad del campo que indica la longitud de la cabecera
- El datagrama de **IPv6 es más simple** que el de IPv4, con menos campos y sin la posibilidad de opciones, por lo que la longitud de la cabecera es fija
- La cabecera de **IPv6 tiene mayor tamaño** que la de IPv4 porque las direcciones IP de origen y destino son de 128 bits en lugar de 32 bits
- Los primeros 4 bits de ambas cabeceras identifican la **versión de IP**, que será 4 o 6, determinando así qué protocolo ha de procesar el datagrama al recibirlo

Funcionalidad común

Resumen

IPv4 e IPv6 tienen el mismo objetivo, por lo que comparten varias de las funcionalidades que se esperan del servicio de red

Funcionalidades

- Reenvían el datagrama hacia su destino, para lo cual precisan una **dirección de destino y también una de origen** (para la respuesta). Lo único que cambia es el tamaño de las direcciones, según estudiamos en el tema de la semana 4
- Ambos cuentan con un campo que permite conocer la **longitud del *payload*** o paquete de datos a transferir
- Con el campo **Protocol / Next hop header** (8 bits) se indica el protocolo al que pertenece el paquete de datos enviado. Códigos habituales son 6 para TCP o 17 para UDP
- El campo **TOS (Type Of Service) / TC (Traffic Class)** ocupa 8 bits que denotan la solicitud de un servicio que se ajusta a unas características concretas
- Con el campo **TTL / Hop limit** (8 bits) se determina el número máximo de saltos antes de que se descarte el datagrama si no llega a su destino

Configuración de cabecera IP - Origen

Inicialización

El host de origen es el encargado de inicializar todos los campos de la cabecera del datagrama IP

- Se asigna la versión al campo **Ver** (4 bits): 4 o 6
- Según el servicio de transporte que haya enviado el segmento, se da al campo **Protocol** (8 bits) el valor que corresponda
- Se introduce el *payload* y se asigna su **longitud** al campo correspondiente (16 bits)
- Se almacenan las **direcciones IP** de origen y de destino
- Se da un valor inicial al campo **TTL** (8 bits)
- En IPv4 se dan valores por defecto a los campos **Identification**, **Flags** y **Fragment Offset** si fuese preciso fragmentar el datagrama
- En IPv4 se calcula la longitud de la cabecera y se asigna a **IHL** (4 bits) en función de que haya o no opciones adicionales

Configuración de cabecera IP - Router

Actualización

La capa de red de los equipos de conmutación obtienen el datagrama desde la capa de enlace, examinan la dirección de destino para reenviarlo por la ruta adecuada e introducen cambios en algunos campos de la cabecera

1. Se toma el campo **TTL** y se reduce en una unidad
2. Si el campo **TTL** es cero, el datagrama se descarta en lugar de ser reenviado
3. Si el datagrama se descarta se envía un paquete de **notificación de error** al origen

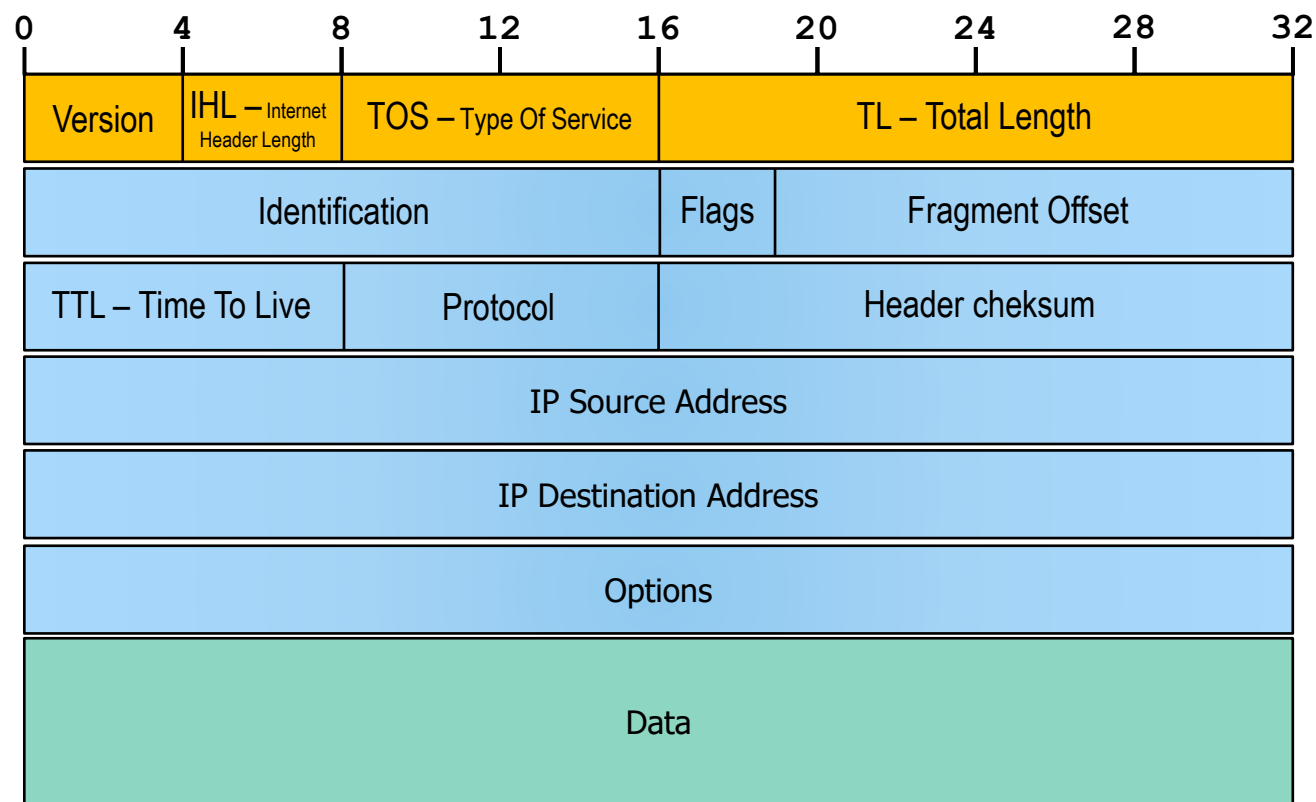
Formato del datagrama

Detalles de IPv4

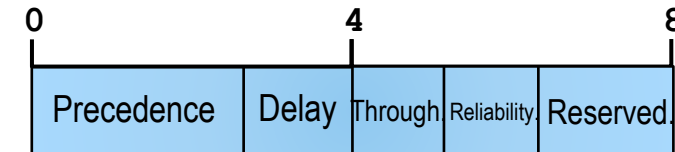


Detalles del datagrama de IPv4

IPv4



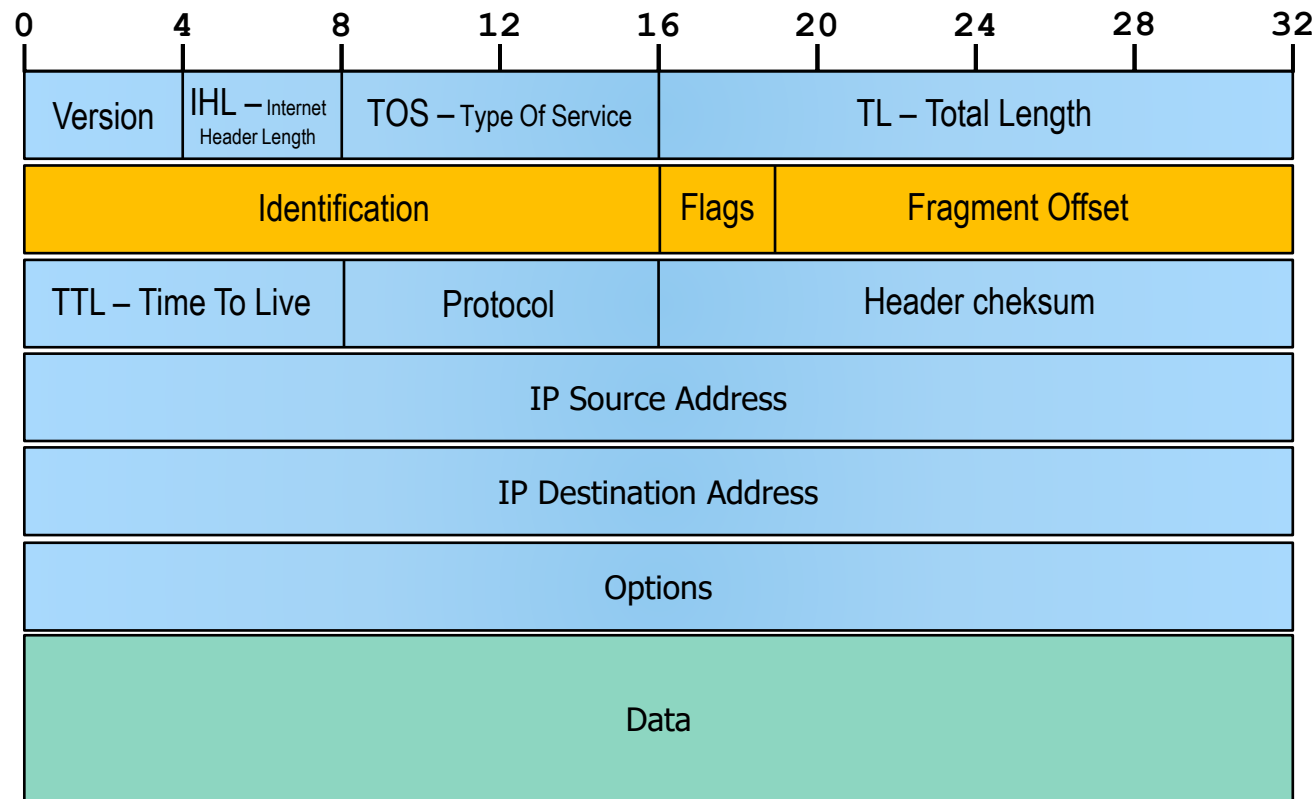
- **Versión** (4 bits): número de versión del protocolo IP del datagrama, 4 para IPv4 y 6 para IPv6
- **IHL - Longitud de cabecera** (4 bits): necesario ya que el número de opciones es variable. Cuando no hay opciones este valor es 5 (20 bytes)
- **TOS - Tipo de servicio** (8 bits): calidad de servicio de transmisión, poco usado. Composición:



- Bits 0-2: indican prioridad del datagrama
 - Bit 3 act. (*D-Delay*): mínimo retardo
 - Bit 4 act. (*T-Throughput*): máximo rendimiento
 - Bit 5 act. (*R-Reliability*): máxima fiabilidad
 - Bits 6-7: uso reservado
- **TL - Longitud** (16 bits): longitud total de todo el datagrama en bytes. Típica entre 500 y 1460 bytes

Detalles del datagrama de IPv4

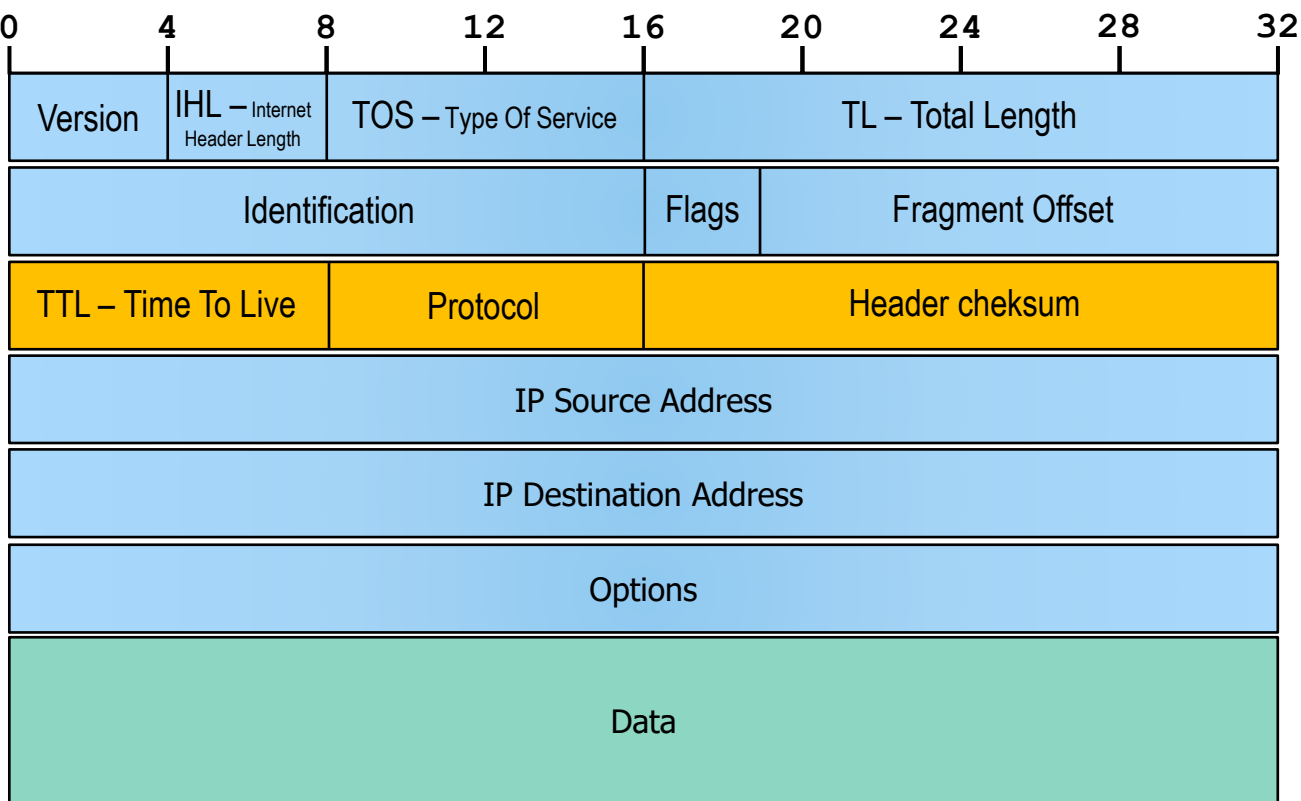
IPv4



- **Identification** (16 bits): identificador único asignado a cada datagrama enviado en transmisión. Si se fragmenta, todos los paquetes mantienen la identificación
- **Flags** (3 bits):
 - Bit 0: no se usa
 - Bit 1: (*Don't fragment*) indica que no se fragmente el datagrama
 - Bit 2: (*More fragments*) indica si hay (1) más fragmentos o no (0)
- **Fragment offset** (13 bits):
 - Se usa para indicar a partir de qué número de byte del datagrama original hay que introducir los bytes que vienen con este fragmento

Detalles del datagrama de IPv4

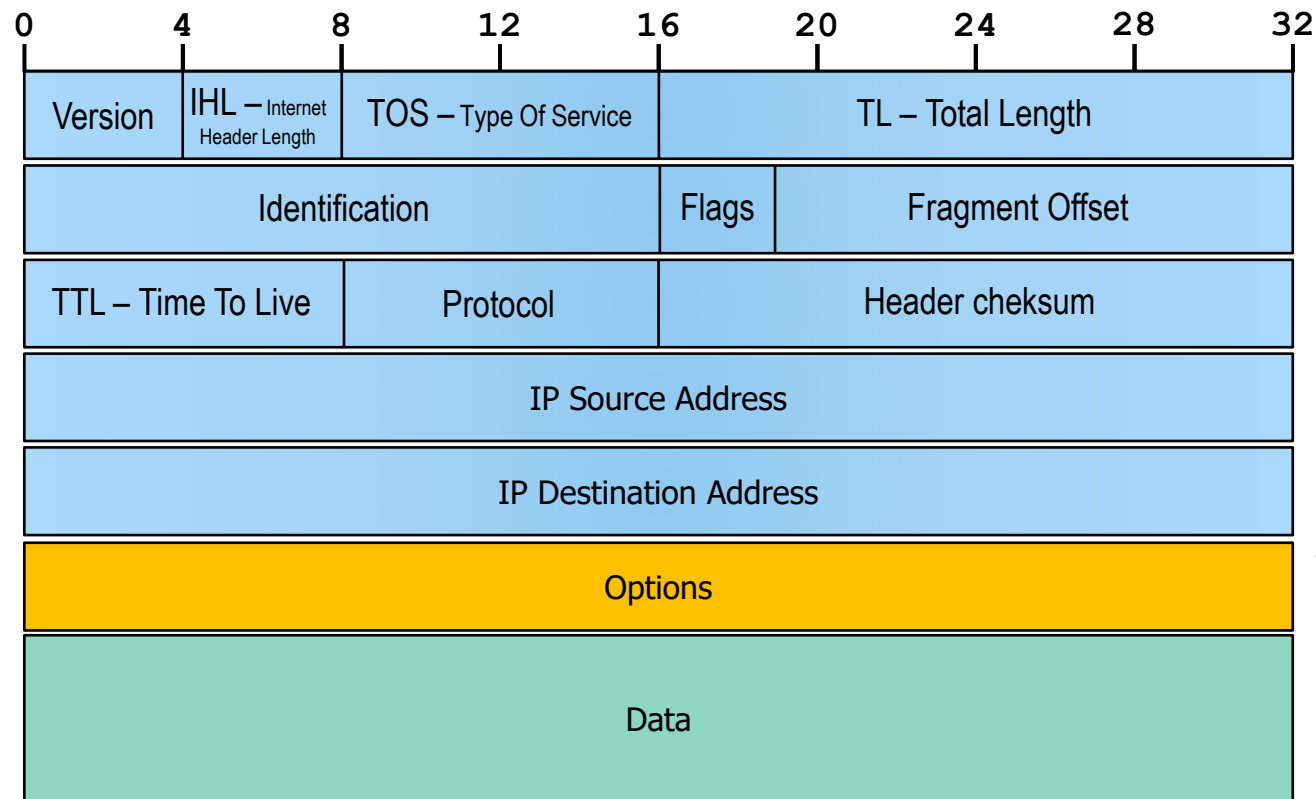
IPv4



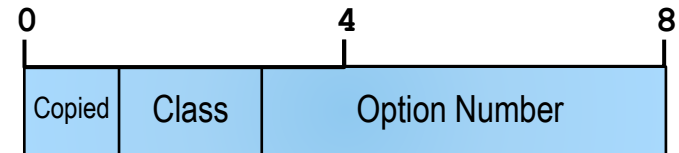
- **TTL – Tiempo de vida** (8 bits): controla que un datagrama no quede eternamente circulando por la red. El emisor le asigna un valor X que va reduciéndose en 1 por cada *router* por el que se pasa hacia el destino
- **Protocol** (8 bits): utilizado en los equipos extremos de la transmisión. Indica el protocolo de la capa de transporte al que va destinado el datagrama. P.e: TCP: 6, UDP: 17
- **Header checksum – Suma de comprobación de la cabecera** (16 bits): sirve para detectar errores en la cabecera. Se calcula tomando los bits de la cabecera en palabras de 16 bits y sumándolas con aritmética en complemento a 1. Normalmente se desechan los datagramas erróneos.

Detalles del datagrama de IPv4

IPv4



- **Options:** la mayoría de datagramas no incluyen opciones para no sobrecargar la cabecera ni a los *router*. Cada opción se precede con una cabecera como la siguiente:



- **Bit 0 - Copied:** indica si la opción debe copiarse a los fragmentos en caso de que el datagrama se fragmentase
- **Bits 1 y 2 - Class:** valor entre 0 y 3 que indica el tipo de opción
 - 0: control de red.
 - 2: depuración y test
 - 1 y 3: reservado para uso futuro
- **Bits 3 a 7 - Option Number:** valor entre 0 y 31 que indentifica la opción a incluir
- Las opciones más destacables son:
 - **Registro de ruta:** sirve para registrar en el datagrama la ruta por la que ha pasado. Los *router* deberán ir rellenando este campo
 - **Encaminamiento desde el origen:** se indica el camino exacto que debe seguir un datagrama. El *router* no mira tablas de enrutamiento sino que mira en este campo donde se debe enviar
 - **Marca de tiempo:** igual que la primera opción pero el *router*, además de grabar su dirección debe grabar el tiempo de paso del datagrama
- Tras la cabecera se alojarían los datos asociados a la opción

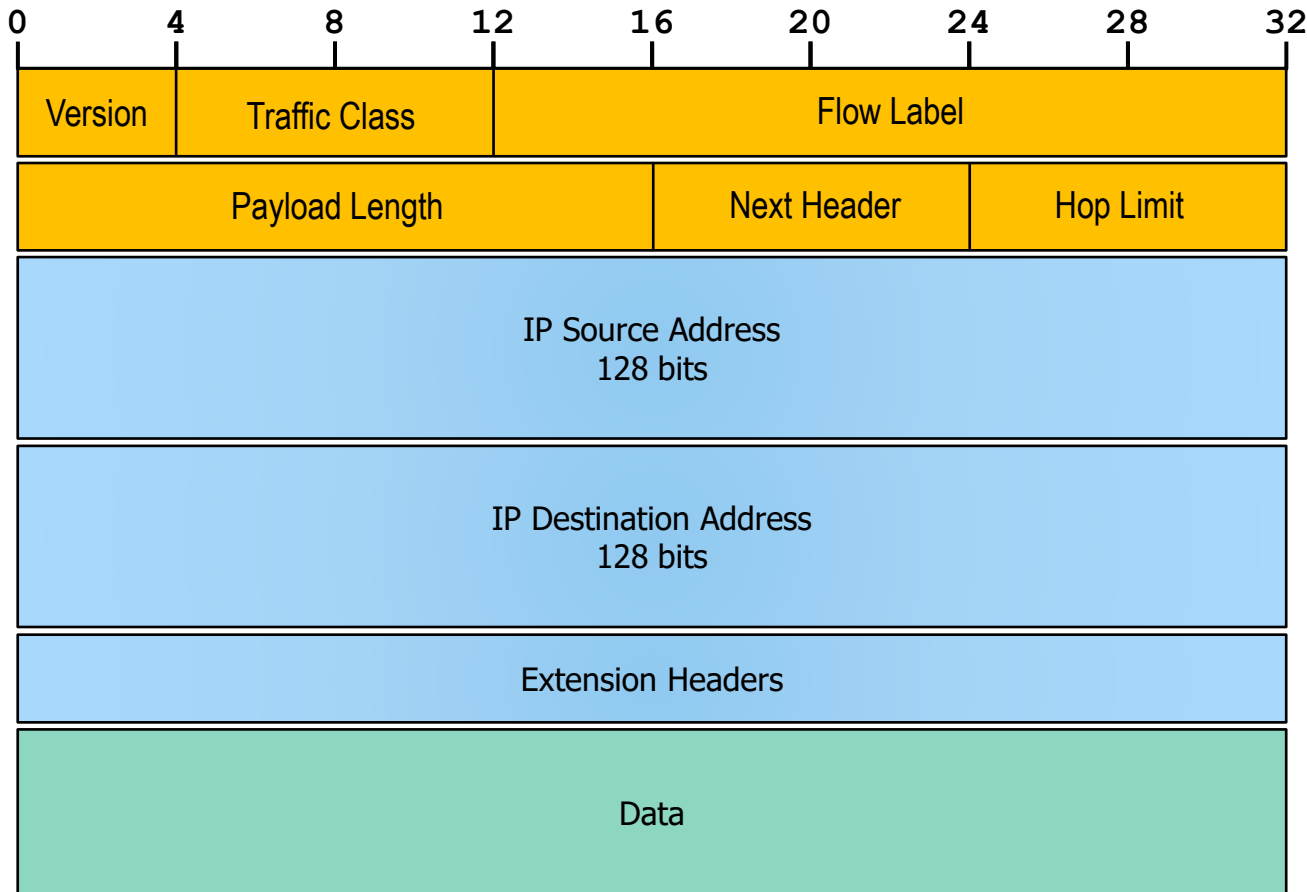
Formato del datagrama

Detalles de IPv6



Detalles del datagrama de IPv6

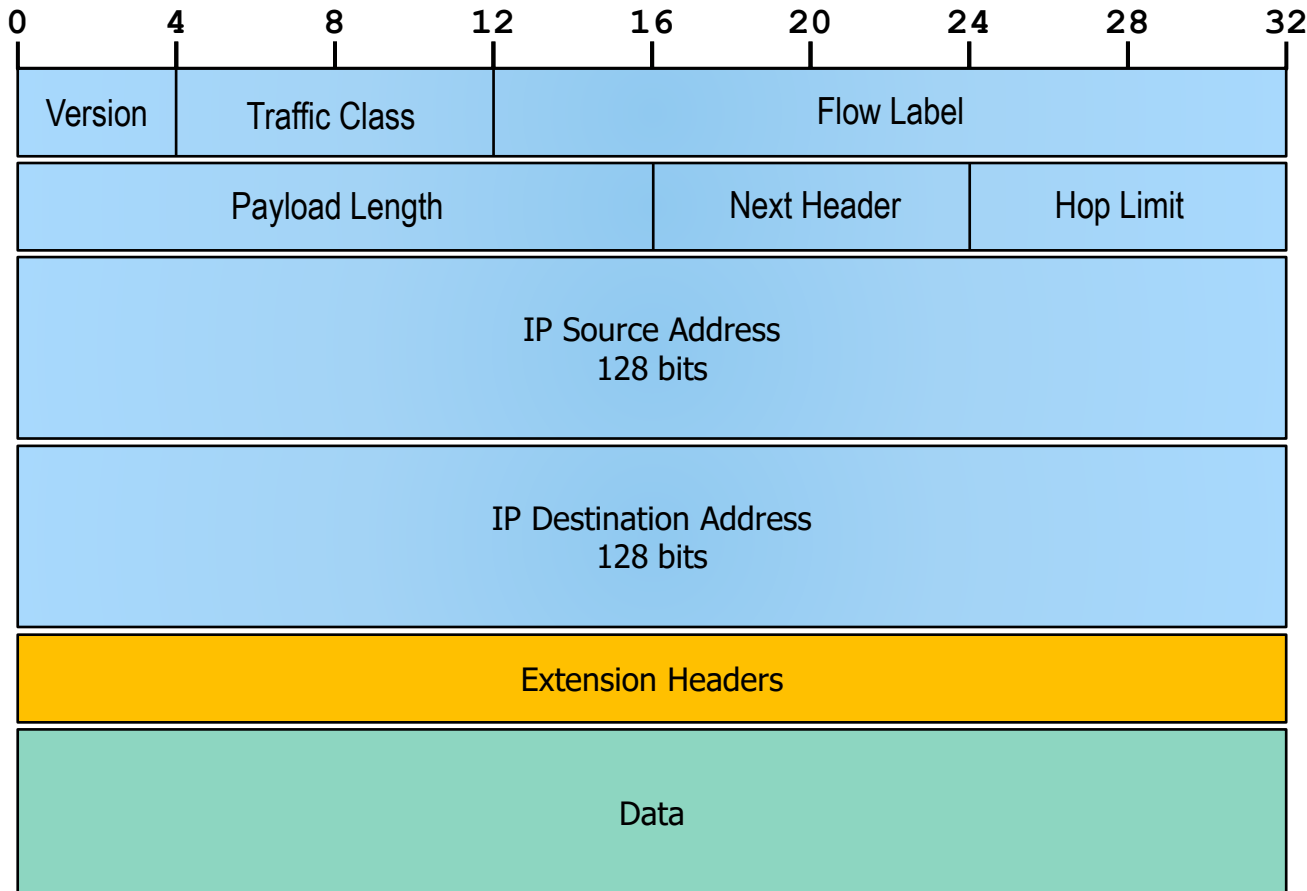
IPv6



- **Version** (4 bits): versión (como en IPv4)
- **Traffic Class** (8 bits): clases de tráfico o prioridades
- **Flow Label** (20 bits): sirve para etiquetar paquetes que pertenezcan a un mismo flujo (*stream*), por ejemplo, de audio o vídeo
- **Payload Length** (16 bits): longitud del campo de datos
- **Next Header** (8 bits): identificación de la cabecera del paquete que se está encapsulando
- **Hop Limit** (8 bits): igual a TTL en IPv4

Detalles del datagrama de IPv6

IPv6



- **Next header** (8 bits): introduce el concepto de cabeceras encadenadas, ampliando las posibilidades respecto a IPv4:

- 4 - IPv4
- 6 - TCP
- 17 - UDP
- 41 - IPv6
- 44 - Cabecera de fragmentación
- 0 - Opciones *hop-by-hop*

(<https://tools.ietf.org/html/rfc2460#page-11>)

Un datagrama IPv6 puede contar con **múltiples cabeceras**, dependiendo de las necesidades, evitando así el uso de los campos menos útiles de IPv4 en la cabecera estándar

Actividades y recursos adicionales



Estructuras de datos asociadas a las cabeceras IP

- Ejecuta en tu máquina virtual el comando `sudo apt install build-essential` a fin de instalar los archivos de cabecera de C/C++ necesarios para desarrollar aplicaciones Linux
- En el directorio `/usr/include/netinet` encontrarás los archivos `ip.h` e `ip6.h`
- Examina dichos archivos (los puedes abrir con el editor nano) y responde a estas cuestiones:
 - ¿Qué tipo de dato se usa para el campo `ttl` de la cabecera de IPv4?
 - ¿Cuál es el tipo de datos para las direcciones IP de origen y destino? ¿Qué diferencia hay entre IPv4 e IPV6?
 - Examina las constantes que se definen a lo largo del archivo de cabecera `ip.h` para saber cuál es el valor por defecto que se asigna al campo TTL. ¿Cuál es el nombre de la constante que almacena dicho valor?
 - ¿Cuál es el MSS por defecto para IPv4 según el archivo de cabecera `ip.h`?

```
usuario@par:~$  
usuario@par:~$ sudo apt install gcc  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias... Hecho  
Leyendo la información de estado... Hecho  
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.  
  dns-root-data dnsmasq-base libbluetooth3 libdbusmenu-glib4 libdbusmenu-gtk3-4 liblvm1 libndp0  
  libnm0 libteamdctl0 net-tools ppp pptp-linux squashfs-tools xul-ext-ubufox  
Utilice «sudo apt autoremove» para eliminarlos.  
Paquetes sugeridos:  
  gcc-multilib make autoconf automake libtool flex bison gdb gcc-doc  
Se instalarán los siguientes paquetes NUEVOS:  
  gcc  
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 0 no actualizados.  
Se necesita descargar 5.212 B de archivos.  
Se utilizarán 51,2 kB de espacio de disco adicional después de esta operación.  
Des:1 http://es.archive.ubuntu.com/ubuntu hirsute/main amd64 gcc amd64 4:10.3.0-1ubuntu1 [5.212 B]  
Descargados 5.212 B en 0s (46,9 kB/s)  
Seleccionando el paquete gcc previamente no seleccionado.  
(Leyendo la base de datos ... 257323 ficheros o directorios instalados actualmente.)  
Preparando para desempaquetar .../gcc_4%3a10.3.0-1ubuntu1_amd64.deb ...  
Desempaquetando gcc (4:10.3.0-1ubuntu1) ...  
Configurando gcc (4:10.3.0-1ubuntu1) ...  
Procesando disparadores para man-db (2.9.4-2) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
usuario@par:~$  
usuario@par:~$ nano /usr/include/netinet/ip.h
```

Número de saltos

- El mismo comando ping también permite, mediante la opción `-t`, fijar el valor inicial del campo TTL
- Usa esa opción con distintos valores y contesta las siguientes cuestiones:
 - ¿Cuál es el mínimo valor de TTL para obtener respuesta de `google.com`?
 - Cuando el valor inicial dado a TTL es inferior al anterior, ¿quién devuelve el paquete con el que se notifica el error?
 - Asumiendo que los paquetes de respuesta de eco pasan por los mismos equipos de conmutación que los de petición, ¿cuál es el valor inicial que está asignando `google.com` al campo TTL del datagrama?

```
[usuario@servidor ~]$  
[usuario@servidor ~]$ ping -t 10 -c 3 fcharte.com  
PING fcharte.com (185.199.108.153) 56(84) bytes of data.  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=1 ttl=59 time=25.2 ms  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=2 ttl=59 time=26.2 ms  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=3 ttl=59 time=23.8 ms  
  
--- fcharte.com ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2007ms  
rtt min/avg/max/mdev = 23.838/25.096/26.212/0.974 ms  
[usuario@servidor ~]$  
[usuario@servidor ~]$ ping -t 5 -c 3 fcharte.com  
PING fcharte.com (185.199.108.153) 56(84) bytes of data.  
From 10.49.62.181 (10.49.62.181) icmp_seq=1 Time to live exceeded  
From 10.49.62.181 (10.49.62.181) icmp_seq=2 Time to live exceeded  
From 10.49.62.181 (10.49.62.181) icmp_seq=3 Time to live exceeded  
  
--- fcharte.com ping statistics ---  
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2007ms  
  
[usuario@servidor ~]$ ping -t 7 -c 3 fcharte.com  
PING fcharte.com (185.199.108.153) 56(84) bytes of data.  
  
--- fcharte.com ping statistics ---  
3 packets transmitted, 0 received, 100% packet loss, time 2006ms  
  
[usuario@servidor ~]$ ping -t 8 -c 3 fcharte.com  
PING fcharte.com (185.199.108.153) 56(84) bytes of data.  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=1 ttl=59 time=24.7 ms  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=2 ttl=59 time=25.4 ms  
64 bytes from cdn-185-199-108-153.github.com (185.199.108.153): icmp_seq=3 ttl=59 time=24.9 ms  
  
--- fcharte.com ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2006ms  
rtt min/avg/max/mdev = 24.732/25.033/25.440/0.298 ms  
[usuario@servidor ~]$ _
```


Indicadores TOS

- Ejecuta el siguiente comando para instalar la herramienta hping3 en tu máquina:
 - `sudo apt update ; sudo apt install hping3`
- Esta utilidad permite configurar el TOS (*Type-Of-Service*) solicitado. Lo habitual es activar únicamente un bit del campo TOS
- Consulta la ayuda de hping3 y, en particular, de la opción `--tos`, para hacer pruebas con distintos valores y responder lo siguiente:
 - ¿Qué representa cada uno de los valores de `--tos`?
 - ¿Hay diferencias en el tiempo de respuesta según el valor asignado a TOS?
 - ¿Cambia el campo TTL en la respuesta según el valor asignado a TOS?
 - En general, ¿crees que tiene alguna incidencia en la comunicación cambiar los bits de TOS?

```
[usuario@servidor ~]$ hping --tos help
tos help:
      TOS Name           Hex Value           Typical Uses
      Minimum Delay      10                  ftp, telnet
      Maximum Throughput  08                  ftp-data
      Maximum Reliability 04                  snmp
      Minimum Cost        02                  nntp

[usuario@servidor ~]$ sudo hping3 -c 5 -S -p 80 --tos 10 google.es
HPING google.es (enp0s3 172.217.168.163): S set, 40 headers + 0 data bytes
len=46 ip=172.217.168.163 ttl=59 id=62881 sport=80 flags=SA seq=0 win=65535 rtt=25.1 ms
len=46 ip=172.217.168.163 ttl=59 id=22832 sport=80 flags=SA seq=1 win=65535 rtt=30.0 ms
len=46 ip=172.217.168.163 ttl=60 id=27571 sport=80 flags=SA seq=2 win=65535 rtt=24.8 ms
len=46 ip=172.217.168.163 ttl=59 id=50676 sport=80 flags=SA seq=3 win=65535 rtt=24.8 ms
len=46 ip=172.217.168.163 ttl=60 id=5601 sport=80 flags=SA seq=4 win=65535 rtt=27.7 ms

--- google.es hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 24.8/26.5/30.0 ms
[usuario@servidor ~]$ sudo hping3 -c 5 -S -p 80 --tos 8 google.es
HPING google.es (enp0s3 172.217.168.163): S set, 40 headers + 0 data bytes
len=46 ip=172.217.168.163 ttl=59 id=26436 sport=80 flags=SA seq=0 win=65535 rtt=29.1 ms
len=46 ip=172.217.168.163 ttl=60 id=31315 sport=80 flags=SA seq=1 win=65535 rtt=28.7 ms
len=46 ip=172.217.168.163 ttl=60 id=21627 sport=80 flags=SA seq=2 win=65535 rtt=29.6 ms
len=46 ip=172.217.168.163 ttl=59 id=34774 sport=80 flags=SA seq=3 win=65535 rtt=30.3 ms
len=46 ip=172.217.168.163 ttl=60 id=55136 sport=80 flags=SA seq=4 win=65535 rtt=25.1 ms

--- google.es hping statistic ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 25.1/28.5/30.3 ms
[usuario@servidor ~]$
```

Fragmentación y MTU

- El comando `ping` nos permite enviar un paquete a otro *host* y solicitar eco, de forma que se obtenga como respuesta un paquete con el mismo contenido
- Esta utilidad cuenta con múltiples opciones que puedes consultar con `man ping`, entre ellas:
 - `-M do`: impide que se fragmenten los datagramas
 - `-s tam`: fija la longitud del paquete de datos
 - `-c veces`: indica el número de repeticiones
- Usando este comando prueba en tu equipo a enviar paquetes de diferentes tamaños, siempre deshabilitando la fragmentación, descubriendo así cuál es la longitud máxima del paquete de datos que puedes enviar
- Cuáles son el MSS y MTU según esas pruebas

```
usuario@par:~$  
usuario@par:~$ ping -M do -s 1460 -c 3 ujaen.es  
PING ujaen.es (150.214.170.66) 1460(1488) bytes of data.  
1468 bytes from wwwr.ujaen.es (150.214.170.66): icmp_seq=1 ttl=61 time=1.72 ms  
1468 bytes from wwwr.ujaen.es (150.214.170.66): icmp_seq=2 ttl=61 time=1.61 ms  
1468 bytes from wwwr.ujaen.es (150.214.170.66): icmp_seq=3 ttl=61 time=1.74 ms  
  
--- ujaen.es ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2005ms  
rtt min/avg/max/mdev = 1.613/1.689/1.736/0.054 ms  
usuario@par:~$  
usuario@par:~$ ping -M do -s 1480 -c 3 ujaen.es  
PING ujaen.es (150.214.170.66) 1480(1508) bytes of data.  
ping: local error: message too long, mtu=1500  
ping: local error: message too long, mtu=1500  
ping: local error: message too long, mtu=1500  
  
--- ujaen.es ping statistics ---  
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2030ms  
usuario@par:~$ _
```


Exploración del datagrama IP



Wireshark

Una de las vías más interesantes para familiarizarse con la estructura de los datagramas IP consiste en capturar el tráfico de nuestra interfaz, por ejemplo con Wireshark, y examinar los datagramas: la versión, longitud de cabecera, longitud total, indicadores, TTL, etc. Todos esos campos, con valores reales y correspondientes a una comunicación real, los tenemos ahí disponibles.

ip.addr==192.168.2.161

No.	Time	Source	Destination	Protocol	Length	Info
133	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=1 Ack=1 Win=9 Len=1460 [T...
134	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=1461 Ack=1 Win=9 Len=1460...
135	0.646287	192.168.2.200	192.168.2.161	TLSv1.2	1252	Application Data
136	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=4119 Ack=1 Win=9 Len=1460...
137	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=5579 Ack=1 Win=9 Len=1460...
138	0.646287	192.168.2.200	192.168.2.161	TLSv1.2	1252	Application Data
139	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=8237 Ack=1 Win=9 Len=1460...
140	0.646287	192.168.2.200	192.168.2.161	TCP	1514	443 → 4193 [ACK] Seq=9697 Ack=1 Win=9 Len=1460...
141	0.646287	192.168.2.200	192.168.2.161	TLSv1.2	721	Application Data

Frame 133: 1514 bytes on wire (12112 bits), 1514 bytes captured
Ethernet II, Src: 9e:05:d6:49:d6:e0 (9e:05:d6:49:d6:e0), Dst: Ir
Internet Protocol Version 4, Src: 192.168.2.200, Dst: 192.168.2.
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 1500
Identification: 0xd687 (54919)
010. = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: TCP (6)
Header Checksum: 0xd7da [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.2.200
Destination Address: 192.168.2.161
[Stream index: 3]
Transmission Control Protocol, Src Port: 443, Dst Port: 4193, S...

Material adicional

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- **Capítulo 4** La capa de red: el plano de datos, del libro Redes de computadoras 7ED disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador), concretamente desde la **sección 4.3** en adelante
- **IP datagram encapsulation and format** en el recurso electrónico [The TCP/IP Guide](#), donde encontrarás información sobre los campos del datagrama y un esquema general del encapsulamiento en IPv4
- **IP datagram size, MTU and fragmentation** en el recurso electrónico [The TCP/IP Guide](#) para conocer en detalle el proceso de fragmentación y reensamblado de datagramas

Cuestiones clave de este tema



Cuestiones clave

Qué deberías saber

*Al inicio de este tema se planteaban unos objetivos específicos que deberían permitirte **responder a las siguientes cuestiones clave***

Cuestiones

- ¿Cómo se encapsulan los segmentos TCP/UDP en un datagrama IP?
- ¿Cuál es la finalidad de los campos más importantes en la cabecera de los datagramas IPv4 e IPv6?
- ¿Qué trabajo realizan los equipos de interconexión sobre el datagrama a medida que lo reenvían?
- ¿Cómo se controla que un datagrama no quede circulando demasiado tiempo sin llegar a su destino?
- ¿Cuál es el mecanismo que permite enviar paquetes de datos mayores que el fijado por el MTU?